Pool Modes are listed below, ordered from the most aggressive level to the most conservative level in how connections are reused. A more aggressive Pool Mode will reuse connections earlier, potentially achieving higher levels of concurrency, at the cost of having to disable certain client side features.

Some features like *Prepared Statements* and *Advisory Locks* are not compatible with all pool modes. These are covered later.

The descriptions below are directly from PgBouncer.

Mode	Description
session	The default mode. The Server is released back to pool after client disconnects.
transaction	Server is released back to pool after transaction finishes.
statement	Server is released back to pool after query finishes. Transactions spanning multiple statements are disallowed in this mode.

Table 4—Pool Modes

Selecting a Pool Mode for maximum reuse

The *Transaction* Pooling Mode cannot be used with Prepared Statements, which are enabled by default in Active Record.

The statement Pool Mode is not commonly used. This was discussed in the *Connection Poolers*[11] episode of Postgres.fm.

The Transaction mode is most commonly used. To use the *Transaction* Pooling Mode, though, Prepared Statements must be disabled.

If disabling Prepared Statements is not a viable path, then the *Session* mode from PgBouncer is the next best option.

Now that you've installed and configured a connection pooler, and have an idea of how it can be used, shift your focus to errors related to database connection.

Identifying Connection Errors and Problems

In this section you'll use psql to analyze connections. Connections may be viewed in PgHero[12] which you began to use earlier.

Some connection errors to avoid are:

- Exhausting connections

11. https://postgres.fm/episodes/connection-poolers
12. https://github.com/ankane/pghero